

Task 2: Diffusion driven instability

We have the reaction dynamics

$$\begin{aligned}\frac{\partial u}{\partial t} &= a - (b+1)u + u^2v + D_u \nabla^2 u, \\ \frac{\partial v}{\partial t} &= bu - u^2v + D_v \nabla^2 v,\end{aligned}$$

where $a, b \geq 0$.

a)

To determine the spatially homogeneous steady state we ignore the diffusion terms and get the system

$$\begin{aligned}\frac{\partial u}{\partial t} &= f(u, v) = a - (b+1)u + u^2v, \\ \frac{\partial v}{\partial t} &= g(u, v) = bu - u^2v.\end{aligned}$$

Setting the partial derivatives to zero we get a system of equations:

$$\begin{aligned}a - (b+1)u + u^2v &= 0 \\ bu - u^2v &= 0\end{aligned}$$

Solving this we get the steady state $u^* = a$, $v^* = \frac{b}{a}$.

To determine the linear stability of the steady state we compute the Jacobian matrix at the fixed point:

$$\mathbf{J}|_{u^*, v^*} = \begin{pmatrix} \frac{\partial f}{\partial u} & \frac{\partial f}{\partial v} \\ \frac{\partial g}{\partial u} & \frac{\partial g}{\partial v} \end{pmatrix} \bigg|_{u^*, v^*} = \begin{pmatrix} 2uv - (b+1) & u^2 \\ b - 2uv & -u^2 \end{pmatrix} \bigg|_{u^*, v^*} = \begin{pmatrix} b-1 & a^2 \\ -b & -a^2 \end{pmatrix}.$$

From this we get $\tau = \text{tr}(\mathbf{J}) = b - 1 - a^2$ and $\Delta = \det(\mathbf{J}) = -a^2(b-1) + a^2b = a^2$. For stability we require that $\tau < 0$, which means that we have stability for $0 < b < 1 + a^2$. At $b = 1 + a^2$ we have discriminant $D = \tau^2 - 4\Delta = -4a^2 < 0$, which means that we have purely imaginary stability exponents. This in turn means that the system undergoes a hopf bifurcation at $b = 1 + a^2$.

b)

With the assumptions $a = 3$, $b = 8$, $D_u = 1$ and $D_v > 1$, we get the Jacobian matrix

$$\mathbf{J} = \begin{pmatrix} 7 & 9 \\ -8 & -9 \end{pmatrix}. \quad (1)$$

Here we have $\tau = \text{tr}(\mathbf{J}) = 7 - 9 = -2 < 0$ and $\Delta = \det(\mathbf{J}) = -63 + 74 = 11 > 0$, which means we have stability for the homogeneous steady state. To determine the stability of the system with diffusion we consider the wave numbers (from lecture 7):

$$k_{\pm}^2 = \frac{1}{2dD_v} \left(dJ_{11} + J_{22} \pm \sqrt{(dJ_{11} + J_{22})^2 - 4d \det \mathbf{J}} \right),$$

where $d = \frac{D_v}{D_u} = D_v$. For stability we require that $dJ_{11} + J_{22} > 0$ and $\frac{(dJ_{11} + J_{22})^2}{4d} > \Delta$. By inserting values for d , J_{11} , J_{22} and Δ we can solve for the critical value D_c where the system becomes unstable due to

diffusion:

$$\begin{aligned}
 7D_c - 9 > 0 &\implies D_c > \frac{9}{7} \approx 1.29 \\
 \frac{(7D_v - 9)^2}{4D_v} = 11 &\implies 49D_c^2 - 126D_c + 81 - 44D_c = 0 \implies \\
 &\implies D_c = \frac{85}{49} \pm \sqrt{\left(\frac{85}{49}\right)^2 - \frac{81}{49}} \approx 1.735 \pm 1.165.
 \end{aligned}$$

The smaller solution is discarded since it doesn't fulfill the first inequality, which gives us the critical diffusion constant $D_c \approx 2.90$ (i.e. we get diffusion driven instability for $D_v > 2.90$).

c)

To simulate the Belousov-Zhabotinsky reaction, we first have to discretize the 2D-Laplacian using

$$\nabla_{\text{discrete}}^2(u) = \frac{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}}{h^2}.$$

An easy implementation that also ensures periodic boundaries is to use `np.roll()` on the grid of states in all four directions. Note that h is set to 1 (the size of a cell in the grid). Then we solve the DE for states u and v using Euler explicit integration:

$$\begin{aligned}
 u_{n+1} &= u_n + dt \cdot (f(u_n, v_n) + D_u \cdot \nabla_{\text{discrete}}^2(u_n)) \\
 v_{n+1} &= v_n + dt \cdot (f(u_n, v_n) + D_v \cdot \nabla_{\text{discrete}}^2(v_n))
 \end{aligned}$$

with $dt = 0.01$. We initialize the 2D grid (with side length $L=128$) as the steady state $u_0 = a$, $v_0 = \frac{b}{a}$, plus a uniformly random perturbation between $\pm 0.1u_0$ and $\pm 0.1v_0$, respectively. Then the simulation is run for $D_v = \{2.3, 3, 5, 9\}$ and for a total of 40000 iterations. Snapshots of the concentration of u at $n = 500$ (transient state) and $n = 40000$ (final state) for each D_v are plotted in the figure below.

From the figure we can see that for $D_v = 2.3$ (which is below our analytically determined critical value $D_c \approx 2.90$), the concentration quickly approaches a spatially homogeneous steady state at $u \approx a = 3$. For the higher diffusion constants, we instead see patterns forming, with more extreme spatial variations (higher peaks and deeper valleys) for higher D_v .

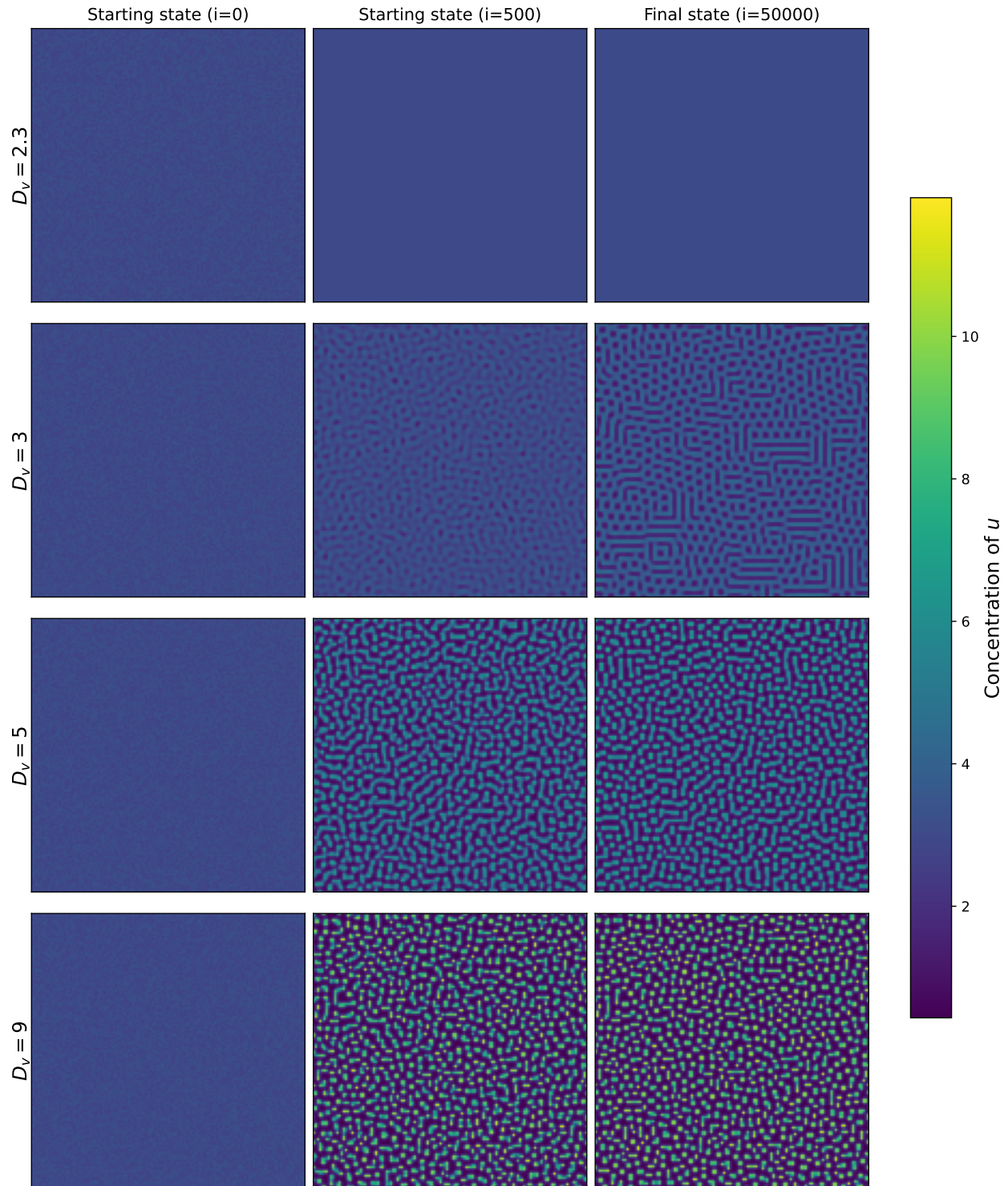


Figure 1: Concentration distribution of u for different D_v and at three different times during simulation.

Python code

```
import numpy as np
```

```

import matplotlib.pyplot as plt

class BelousovDiffusionModel:
    def __init__(self,
        L: int,
        a: float,
        b: float,
        Du: float,
        Dv: float,
        dt: float,
    ) -> None:
        self.L: int = L
        self.a: float = a
        self.b: float = b
        self.Du: float = Du
        self.Dv: float = Dv
        self.dt: float = dt

        self.h = 1 # Step size in grid

        self.init_grid()

    def init_grid(self) -> None:
        initial_state = self.analytic_stable_state()

        initial_grid = np.ones(shape=(self.L, self.L, 2)) * initial_state[None, None, :]

        perturbation = np.zeros(initial_grid.shape)

        bound1 = initial_state[0]*0.1
        bound2 = initial_state[1]*0.1

        perturbation[:, :, 0] = np.random.uniform(-bound1, bound1, size=(self.L, self.L))
        perturbation[:, :, 1] = np.random.uniform(-bound2, bound2, size=(self.L, self.L))

        self.state = initial_grid + perturbation

    def analytic_stable_state(self) -> np.ndarray:
        """
        -----Analytically derived solution when diffusion term is neglected
        -----"""
        return np.array([self.a, self.b / self.a])

    def calc_laplace_discrete(self, state: np.ndarray) -> np.ndarray:
        """
        -----Laplace discretation:

        -----Nabla^2(u) = (u(i+1,j) + u(i-1,j) + u(i,j+1) + u(i,j-1) - 4u(i,j)) / eps^2

        -----np.roll() ensures periodic boundaries
        -----"""
        return (
            np.roll(state, 1, axis=0) + np.roll(state, -1, axis=0) +
            np.roll(state, 1, axis=1) + np.roll(state, -1, axis=1) -

```

```

        4*state) / self.h**2
    )

    def F(self, u: np.ndarray, v: np.ndarray) -> np.ndarray:
        """
        -----Dynamics for u without diffusion
        -----"""
        return self.a - (self.b + 1)*u + u**2*v

    def G(self, u: np.ndarray, v: np.ndarray) -> np.ndarray:
        """
        -----Dynamics for v without diffusion
        -----"""
        return self.b*u - u**2*v

    def update(self) -> None:
        """
        -----Euler explicit numerical integration:
        -----y_(i+1) = y_i + dt*f'(t, y_i)
        -----"""
        u = self.state[:, :, 0]
        v = self.state[:, :, 1]

        Lu = self.calc_laplace_discrete(u)
        Lv = self.calc_laplace_discrete(v)

        u_new = u + self.dt*(self.F(u, v) + self.Du*Lu)
        v_new = v + self.dt*(self.G(u, v) + self.Dv*Lv)

        self.state[:, :, 0] = u_new
        self.state[:, :, 1] = v_new

# Constants
L = 128
A = 3
B = 8
DU = 1
dt = 0.01

Dvs = [2.3, 3, 5, 9]

N_tot = 50000
N_transient = 500

starting_states = []
transient_states = []
final_states = []

# Run simulations
for ax_i, Dv in enumerate(Dvs):
    model = BelousovDiffusionModel(L, A, B, DU, Dv, dt)
    starting_states.append(model.state[:, :, 0].copy())

```

```

for i in range(N_tot):
    model.update()

    if i == N_transient:
        transient_states.append(model.state[:, :, 0].copy())

    final_states.append(model.state[:, :, 0].copy())

# Find max and min values to ensure same color scaling
for state in starting_states + transient_states + final_states:
    v_max = np.max(state)
    v_min = np.min(state)

# Plotting
fig, axes = plt.subplots(4, 3, figsize=(10, 12), constrained_layout=True)

for starting_state, transient_state, final_state, Dv, ax_row in zip(starting_states, tran
    ax_row[0].imshow(starting_state, cmap="viridis", vmin=v_min, vmax=v_max)
    ax_row[1].imshow(transient_state, cmap="viridis", vmin=v_min, vmax=v_max)
    ax_row[2].imshow(final_state, cmap="viridis", vmin=v_min, vmax=v_max)

row_labels = [r"$D_v=2.3$", r"$D_v=3$", r"$D_v=5$", r"$D_v=9$"]
col_labels = [r"Starting-state-(i=0)", f"Starting-state-(i={N_transient})", f"Final-state-(i={N_transient})"]

for ax, row_label in zip(axes[:, 0], row_labels):
    ax.annotate(row_label, xy=(0, 0.5), xycoords='axes-fraction', va='center', ha='right',

for ax, col_label in zip(axes[0, :], col_labels):
    ax.set_title(col_label)

for ax in axes.flatten():
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)

im = axes[ax_i, 2].imshow(final_state, cmap="viridis", vmin=v_min, vmax=v_max)
cbar = fig.colorbar(im, ax=axes, shrink=0.7)
cbar.set_label(r"Concentration-of-$u$", fontsize=14)
fig.savefig("2_2_c.pdf")

```